

Dónde deployar una app de Python

De contratar un hosting a elegir dónde publicar lo que programás

Seminario de Actualización · Tecnicatura Superior en Ciencia de Datos e IA · IFTS N.º 18

Docente: Cynthia M. Villagra · Ciclo lectivo 2026 · Segundo cuatrimestre

Clase 5 — Material de referencia

Parte 1 · De la página web al servidor de Python

Lo que ya conocés: contratar un hosting

Contratás un hosting para una página web y te dan un paquete bastante parecido siempre: un panel de

control, un dominio, espacio en disco, **un servidor de mail** para tener tu casilla

vos@tudominio, **una base de datos MySQL** y un certificado SSL. Subís los archivos y la página está en internet.

Funciona perfecto. Para lo que está diseñado.

Qué es un stack tecnológico

Un **stack** —o pila— es el conjunto de lenguajes, frameworks, bases de datos y herramientas que se combinan para que una aplicación funcione. Se llama pila porque se organiza **en capas**, y cada capa se ocupa de una cosa.

Capa	De qué se ocupa	Ejemplos
Frontend	lo que el usuario ve y toca en su pantalla	HTML, CSS, JavaScript
Backend	la lógica: procesa datos, decide, conecta	Python, Node.js, Java, PHP
Base de datos	dónde se guarda la información	MySQL, PostgreSQL, MongoDB
Infraestructura	dónde corre todo para estar disponible en internet	Linux, Render, AWS

Algunas combinaciones se usan tanto que tienen nombre propio:

- **LAMP** — Linux + Apache + MySQL + PHP
- **MERN** — MongoDB + Express + React + Node.js, todo en JavaScript

****Las tres primeras capas las venís trabajando desde el principio de la cursada. Esta clase es sobre la cuarta.**** Gradio te resolvió frontend y backend en un solo proceso; lo que falta decidir es dónde vive.

Por qué ese hosting no sirve para Python

El hosting compartido que contratás viene con Apache, MySQL y PHP listos porque ****está armado para**

el stack LAMP**. No es que sea malo: está hecho para otra pila.

El problema de fondo es cómo ejecuta el código:

*En ese hosting, cada visita **levanta un script PHP, lo ejecuta y lo mata**. No queda nada corriendo entre un pedido y el siguiente. Tu app de Gradio necesita exactamente lo contrario: **un proceso que quede vivo**, sosteniendo la cola y la conexión abierta con el navegador.*

Y además: no tenés terminal, no podés instalar las dependencias que quieras con pip, y no controlás el sistema.

Un matiz que conviene tener claro: el servidor de mail y el de base de datos de ese hosting **siguen sirviendo perfectamente**. El que no sirve es el de aplicación. Tu app corriendo en Render puede usar la base MySQL de ese mismo hosting sin problema.

Entonces, ¿un VPS? Y por qué sale caro

Un **VPS** (Virtual Private Server) es una máquina virtual entera para vos, con acceso de administrador. Instalás Python, las dependencias, el servidor web, el certificado HTTPS. Todo funciona, porque todo lo ponés vos.

Sale caro por **dos motivos distintos**, y el segundo es el que sorprende:

- **En plata:** pagás la máquina las 24 horas, esté o no usándose. No hay plan gratuito real. Para un portfolio que alguien abre una vez por mes, es desproporcionado.
- **En tiempo:** la administrás vos. Actualizaciones de seguridad, configurar el servidor web, renovar certificados, mirar que no se caiga. **Ese es el costo que nadie te cuenta**, y a la larga suele ser el más caro de los dos.

Un VPS es la respuesta correcta cuando necesitás control total. No cuando querés publicar un ejercicio de la facultad.

Parte 2 · Los tipos que sí sirven

PaaS — Platform as a Service

Le das el repositorio y la plataforma se ocupa del resto: detecta que es Python, instala las dependencias, lo corre y lo expone con HTTPS. **No tocás el sistema operativo.**

Ejemplos: Render, Railway, Heroku, PythonAnywhere, Hugging Face Spaces.

Acá vive el plan gratuito que sirve para un portfolio. Es el tipo que buscás.

Serverless — o FaaS

No hay ningún proceso tuyo corriendo. Subís una función; la plataforma la despierta cuando llega un pedido, la ejecuta y la mata. Pagás por ejecución, no por tiempo encendido.

Ejemplos: Vercel Functions, AWS Lambda, Google Cloud Functions.

Es el caso extremo de "no hay proceso vivo", y por eso **no sirve para Gradio**. Excelente para una API que responde y se olvida; inservible para algo que mantiene estado o una conexión abierta.

Nubes elásticas — AWS, Azure, Google Cloud

No son un tipo de hosting: son proveedores que venden todos los tipos. Esto confunde a todo el mundo y conviene tenerlo claro.

Lo que querés	En AWS	En Google Cloud
VPS	EC2	Compute Engine
PaaS	App Runner	App Engine
Serverless	Lambda	Cloud Functions

"Elástico" quiere decir que los recursos crecen y decrecen solos según la demanda, y pagás por uso. Suena ideal, y para una empresa lo es.

La contracara: sin un límite configurado, **la factura también es elástica**. Tienen planes gratuitos, pero casi siempre piden tarjeta y el riesgo de un cobro por descuido es real. Por eso no son la opción para un portfolio de estudiante.

Resumen

Tipo	Vos ponés	La plataforma pone	¿Gradio?
Hosting compartido	archivos	todo lo demás	✗ no deja procesos vivos
VPS	sistema operativo, Python, servidor web, HTTPS, actualizaciones	la máquina virtual	✓ pero lo mantenés vos
PaaS	el repositorio	SO, runtime, deploy, HTTPS	✓ y acá está el free tier
Serverless	una función suelta	todo, y solo mientras corre	✗ no hay proceso vivo
Nube elástica	depende del servicio	vende los cuatro anteriores	✓ pero pide tarjeta

Parte 3 · Dónde se puede deployar Python

Panorama general, sin filtrar por precio todavía.

Plataforma	Tipo	Plan gratuito
Render	PaaS	sí, sin tarjeta
Railway	PaaS	crédito mensual muy ajustado
Fly.io	PaaS / contenedores	limitado, pide tarjeta
PythonAnywhere	PaaS	sí, pero solo WSGI

Hugging Face Spaces	PaaS especializada en ML	restringido para cuentas nuevas
Streamlit Community Cloud	PaaS, solo Streamlit	sí
Heroku	PaaS	ya no tiene plan gratuito
Replit	PaaS / entorno online	sí, pero la app no queda siempre encendida
AWS / Azure / Google Cloud	de todo	sí, con tarjeta y con riesgo de cobro
DigitalOcean, Linode, Hetzner	VPS	no

Parte 4 • Dónde puedo deployar Gradio gratis

Del panorama de arriba, filtrando por lo que esta materia necesita —**Gradio, gratis y sin tarjeta**— queda poco:

Plataforma	¿Sirve?	Por qué
Render	Sí	Proceso persistente, soporta ASGI/WebSockets, gratis sin tarjeta, deploy desde GitHub
Hugging Face Spaces	No, para cuentas nuevas	Dejó de permitir Spaces Gradio en CPU Basic para cuentas nuevas
Vercel	No	Serverless: no mantiene un proceso vivo
Railway	Ajustado	El plan gratuito da muy poco margen
Replit	No gratis	"Always On" dejó de estar en el plan gratuito
PythonAnywhere	No, en el plan gratuito	No soporta ASGI/WebSockets

Datos de investigación propia de la cátedra, al 16/09/2026.

Los planes gratuitos cambian, y cambian sin avisar. Esta misma cursada lo vivió: Hugging Face Spaces era la opción de la Clase 4 y dejó de servir para cuentas nuevas de un día para el otro.

Por eso lo que importa no es memorizar cuál sirve hoy, sino **saber hacer las preguntas**. La tabla caduca; el checklist que sigue, no.

Parte 5 • El checklist — 10 preguntas antes de elegir

Ordenadas: las primeras tres descartan, el resto define si te va a alcanzar.

1. ¿Tu app necesita un proceso que quede corriendo?

La pregunta que más plataformas descarta de entrada. Si tu app mantiene estado, una cola, una sesión

o una conexión abierta, **necesitás proceso persistente** y el serverless queda afuera.

2. ¿Soporta ASGI, o solo WSGI?

WSGI es sincrónico, un pedido a la vez. **ASGI** es asíncrono y soporta WebSockets. Gradio usa WebSockets para su cola y para que la interfaz se actualice sola: si la plataforma solo soporta WSGI, tu app **no va a funcionar bien** aunque llegue a arrancar.

3. ¿Pide tarjeta de crédito para el plan gratuito?

Un "gratis" que pide tarjeta no es lo mismo que uno que no la pide. Define si podés probar sin exponerte a un cobro por descuido.

4. ¿Cuánto te dan gratis, y en qué unidad?

No alcanza con "tiene plan gratuito". Mirá en qué se mide, porque cambia todo:

Unidad	Qué preguntar
RAM	¿Entra tu app? Un modelo de ML cargado puede comerse 512 MB solo
CPU	Fracciones (0.1, 0.5) significan que va a andar lento, no que no va a andar
Horas por mes	¿Se agota a mitad de mes y tu app se apaga?
Créditos	Se consumen; cuando se acaban, se acaba
Build minutes	Cada deploy consume; si iterás mucho, se agotan

5. ¿Se duerme? ¿Cuánto tarda en despertar?

Muchos planes gratuitos suspenden la app sin tráfico, y el primer visitante espera el *cold start*.

Para un portfolio que alguien abre de vez en cuando esto importa: la primera impresión puede ser una

pantalla en blanco durante un minuto.

6. ¿El disco persiste, o se borra?

El que más sorprende. En la mayoría de los planes gratuitos el sistema de archivos es

efímero: lo que tu app escribe en disco desaparece al reiniciarse o al hacer un deploy nuevo.

Consecuencia directa: si guardás datos, **no los guardes en un archivo local**.

7. ¿Se conecta a una base de datos externa?

- ¿La plataforma ofrece una base propia? ¿Está en el plan gratuito?
- Si usás una externa (Supabase, Neon, MongoDB Atlas), ¿permite salida a internet?
- ¿La base también se suspende por inactividad? Varios planes gratuitos de bases se duermen a los

días sin uso.

8. ¿Cómo maneja los secretos?

Tu API key y tu contraseña de base **no van en el repositorio**. La plataforma tiene que ofrecer **variables de entorno** configurables desde su panel, fuera del código. Si no te deja, no la uses para nada que tenga credenciales.

9. ¿Deploya solo desde GitHub?

Lo ideal: conectás el repositorio y cada push redeploya. Si hay que subir archivos a mano, vas a dejar de actualizarlo a la semana.

10. ¿Qué pasa cuando crezca?

Mirá el primer escalón pago: cuánto cuesta y qué da. Y sobre todo, **¿podés migrar?** Una app que corre en un contenedor estándar se muda; una atada a servicios propietarios de una plataforma, no.

Fuentes

- Gradio — *Understanding Gradio Share Links*:

<https://gradio.app/guides/understanding-gradio-share-links>

- Gradio — *Sharing Your App*: <https://www.gradio.app/guides/sharing-your-app>
- Render — documentación de planes y límites: <https://render.com/docs>
- InvGate — *¿Qué es un stack tecnológico?*: <https://blog.invgate.com/es/stack-tecnologico>
- Aliantec — *Qué es un stack tecnológico y cómo elegir el ideal para tu carrera*:

<https://www.aliantec.com/2025/01/24/que-es-un-stack-tecnologico-y-como-elegir-el-ideal-para-tu-carrera/>

Los datos de plataformas son de investigación propia de la cátedra al 16/09/2026. ****Verificá contra la documentación oficial de cada plataforma antes de decidir****: los planes gratuitos se modifican seguido.